Science and Technology Facilities Council

CCP-NTH CHAPSim2 training: GPU acceleration

Bo Liu*, **Wei Wang**

Scientific Computing (SC),
Science and Technology Facilities Council (STFC),
UKRI, UK
* Correspondence: bo.liu@stfc.ac.uk

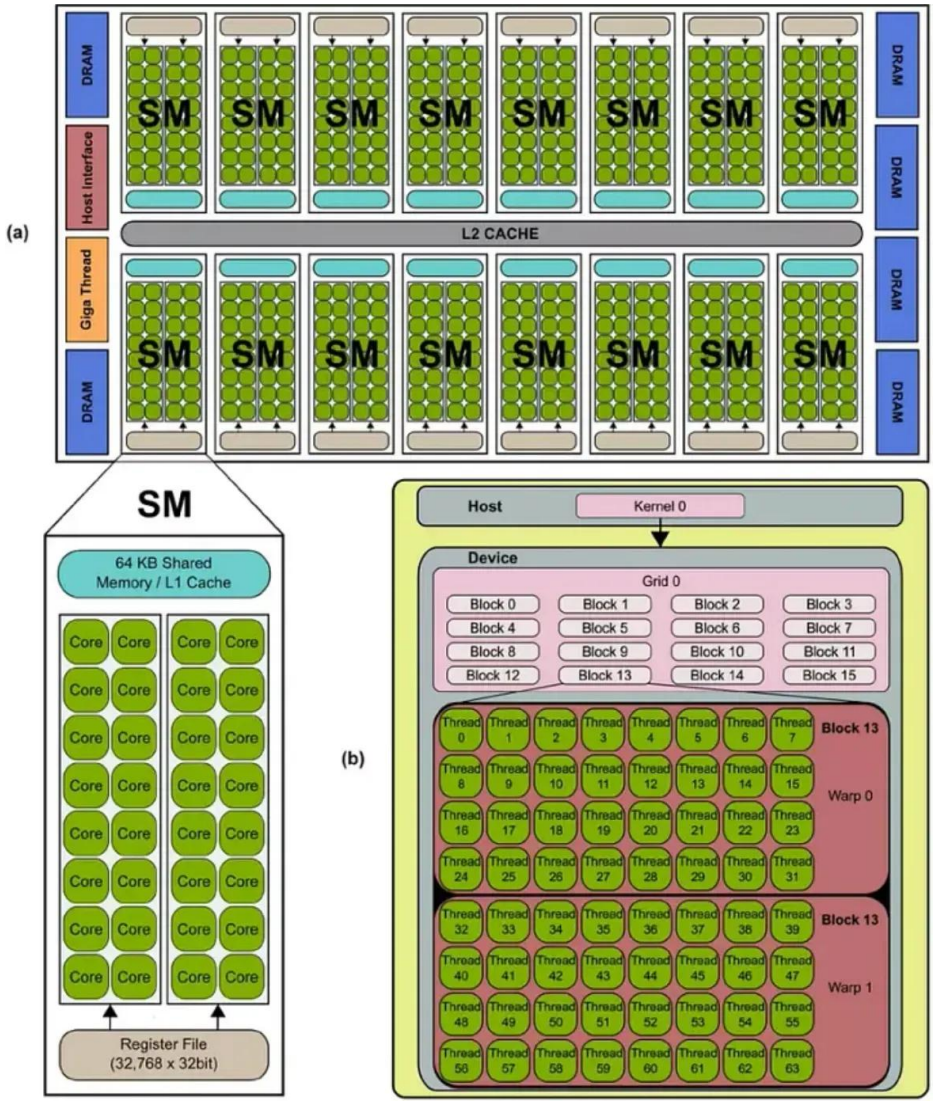
GPU Fundamentals

Key features

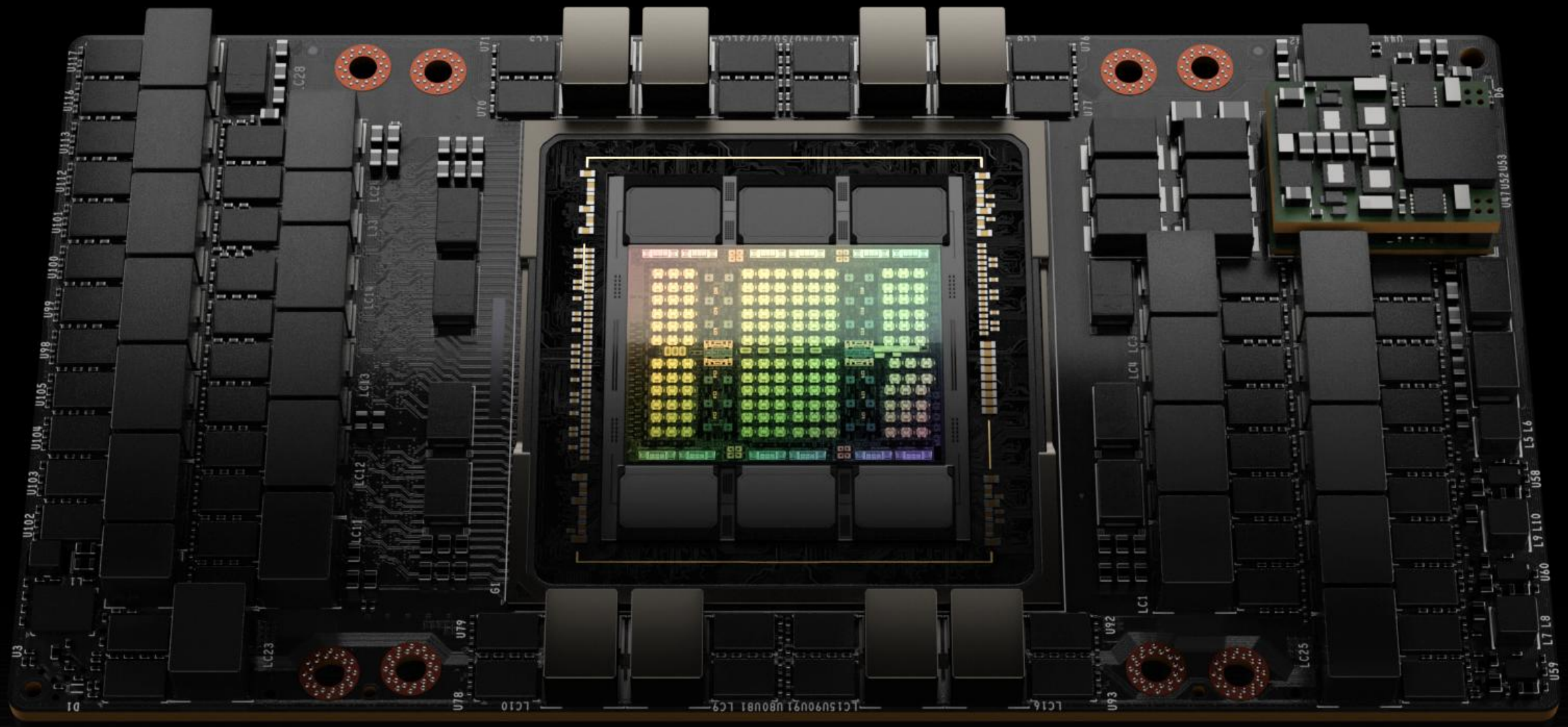
- GPU = massively parallel processor
- Built from Streaming Multiprocessors (SMs)
- Each SM contains:
  - CUDA cores (FP32/FP64)
  - Tensor cores (AI / mixed precision)
  - Register + shared memory

Memory hierarchy

- Registers → fastest, per-thread
- Shared memory (L1 cache) → per-block, software managed
- L2 cache → global on-chip
- HBM (High Bandwidth Memory) → large but slower



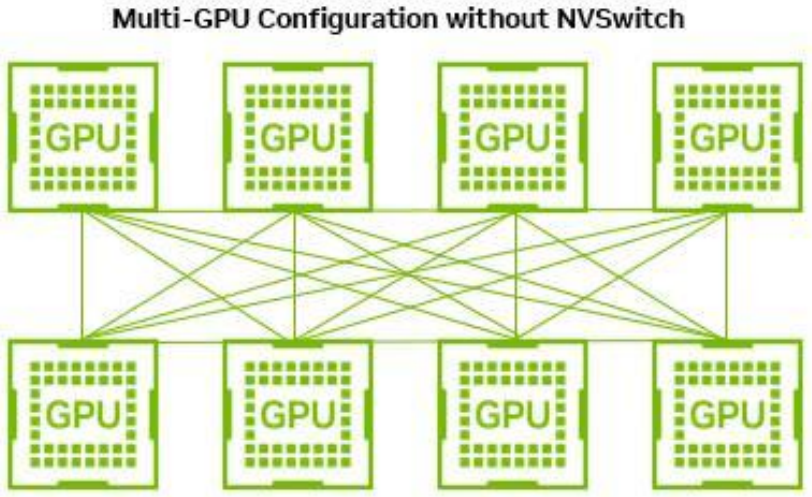
a) Physical Components | b) Logical Components

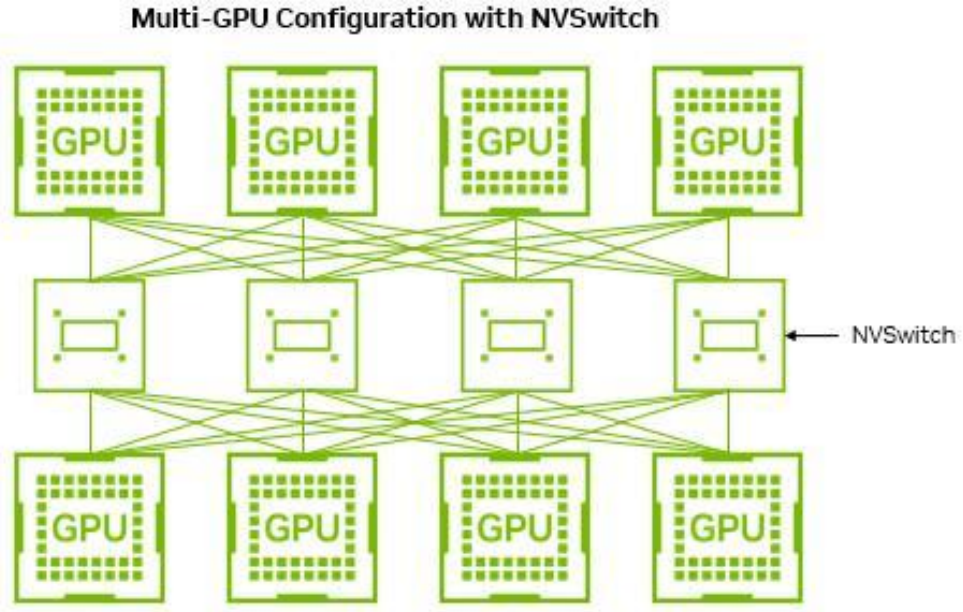


Multi-GPU Node

- **Inside one HPC node**
- 4 - 8 GPUs connected via:
  - NVLink (direct GPU↔GPU)
  - NVSwitch (all-to-all connectivity)
- **Bandwidth**
- NVIDIA A100 NVLink: ~600 GB/s
- NVIDIA H100 NVLink: ~900 GB/s

GPU Count	Point-to-Point Bandwidth	NVSwitch Bandwidth
2	128 GB/s	900 GB/s
4	3 x 128 GB/s	900 GB/s
8	7 x 128 GB/s	900 GB/s

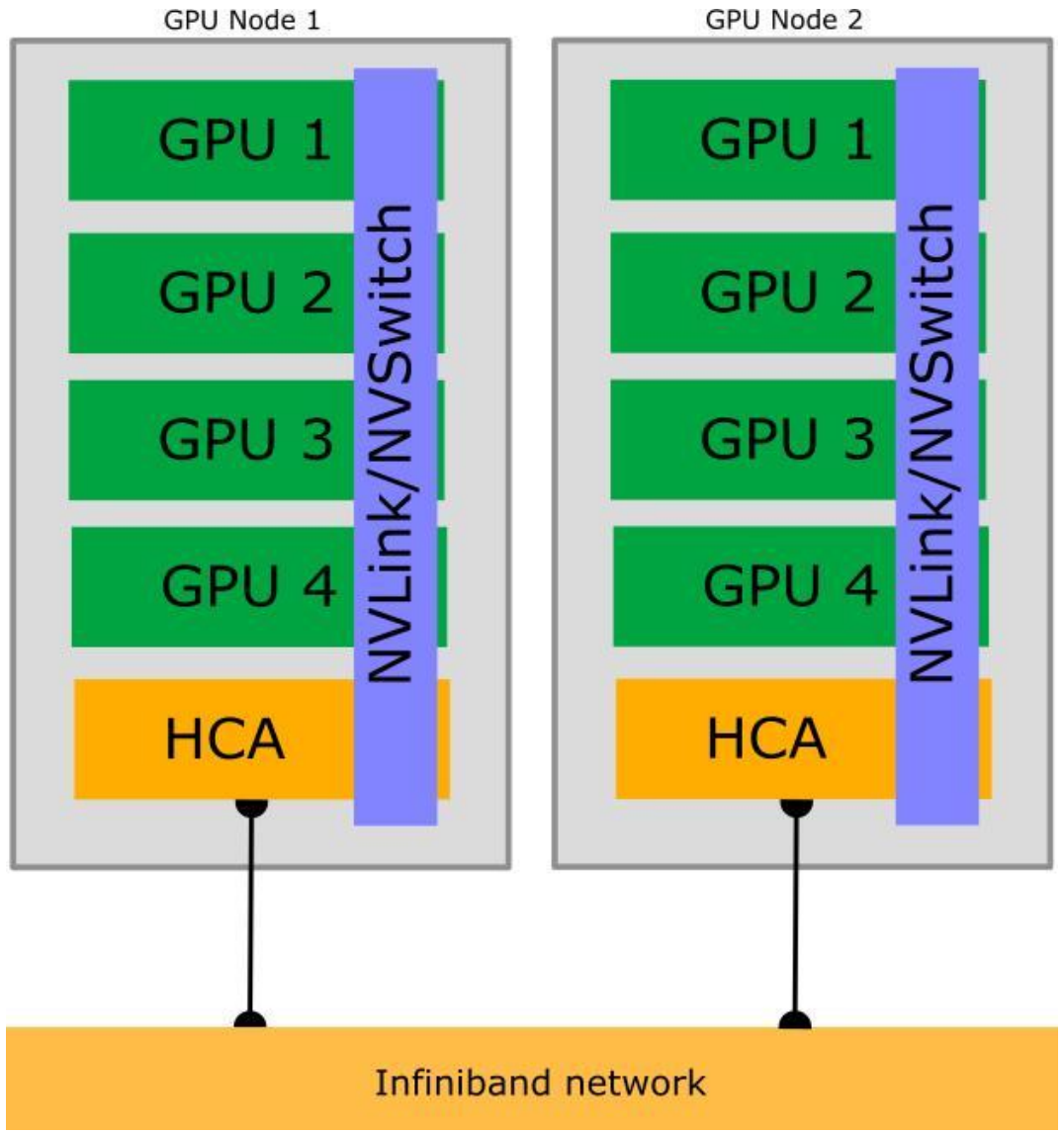




Cluster level

- **Between nodes:**

- Communication via **InfiniBand**
- Typical speeds:
  - FDR: 55 Gbps (~7 GB/s)
  - HDR: 200 Gbps (~25 GB/s)
  - NDR: 400 Gbps (~50 GB/s)
  - 10–20× slower than NVLink
  - 100× slower than on-chip memory



Data movement speeds

Level	Location	Bandwidth (A100)	Bandwidth (H100)	Latency	Notes
Registers	On SM	~20–30 TB/s	~30+ TB/s	🔹 Lowest	Per-thread
Shared Memory	On SM	~10 TB/s	~15 TB/s	🔹 Very low	Manual use
L2 Cache	On GPU	~2–4 TB/s	~4–6 TB/s	Low	Shared
HBM	GPU memory	**1.5–2 TB/s**	**~3.3 TB/s**	Medium	Main working memory
PCIe Gen4/5	CPU↔GPU	~32–64 GB/s	~64–128 GB/s	High	**Bottleneck**
NVLink	GPU↔GPU	~600 GB/s	~900 GB/s	Higher	Intra-node
InfiniBand	Node↔Node	**~25–50 GB/s**	**~50 GB/s**	🔴 Very high	**Bottleneck**

📌 **Key Takeaway:**
Performance depends more on data movement than raw FLOPs

Potential bottleneck

Memory Capacity

- Memory size is still relatively limited on GPUs:
  - A100: 40–80 GB
  - H100: 80–94 GB
- GPU prefers large workloads (> **10 – 50 million** grid points), otherwise:
  - High kernel launch overheads kills performance
  - Low occupancy cannot sufficiently hide memory latency

Memory Bandwidth

- CFD is typically bandwidth-bound (The "Memory Wall"):
  - The arithmetic intensity (math per byte of data) is low
  - GPU often waits for data from HBM
  - FLOPs available >> FLOPs usable

Memory Access Patterns

- Contiguous memory access = fast
- Strided/random memory access = slow

Communication

- MPI + InfiniBand:
  - Bandwidth limits large movement of data
- For DNS:

  Time depends on *"max(Compute, Memory, Communication)"*

$$T \approx T_{HBM} + T_{NVLink} + T_{MPI}$$

CHAPSim2 GPU porting

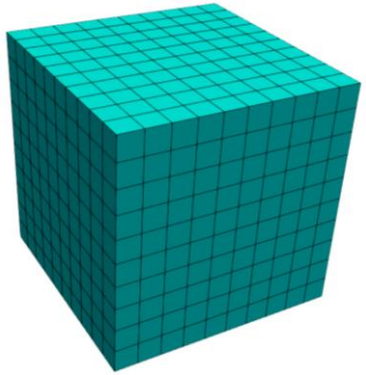
- **Key rules:**
  - Increase memory efficiency
  - Move data as little as possible and as locally as possible

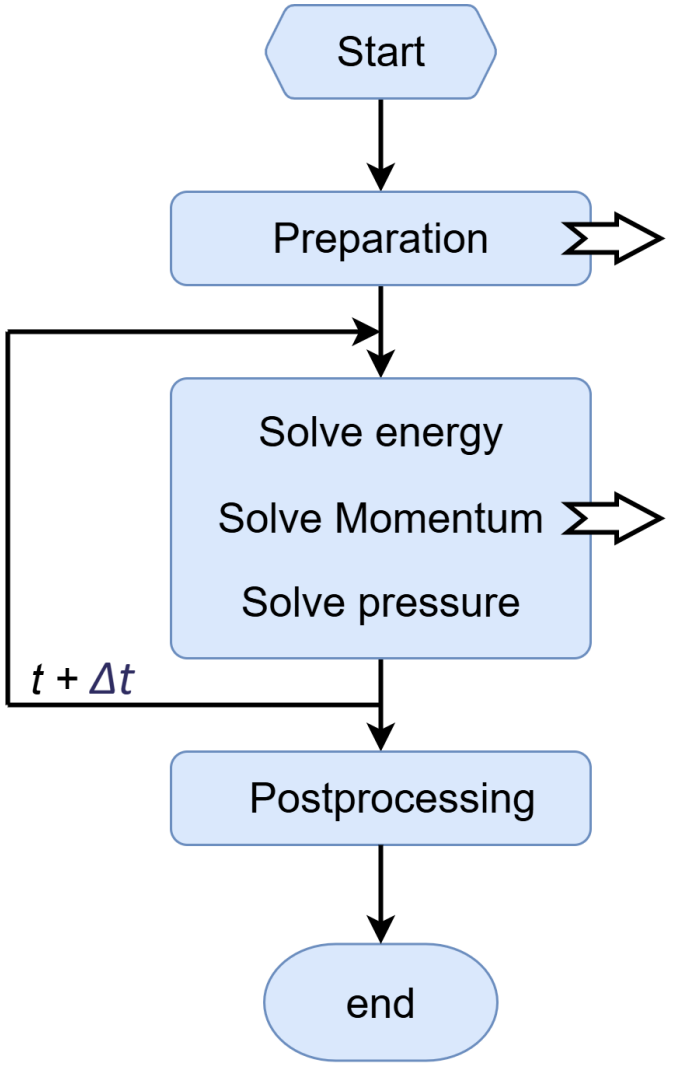
- **Code refactoring strategy:**
  - Replace large number of 3-D local arrays with 1-D re-usable workspace buffers
  - Alias via pointers to get required array shapes
  - Recompute intermediates where needed

N×M×L array

Buffer size:
(N+1)×(M+1)×(L+1)

ptr (1:N, 1:M, 1:L) →





Allocate global arrays:
1. For solver: u, v, w, p, p', h, rhs, etc. (about **12** for isothermal flows)
2. For post processing and I/O: pr_mean, u_mean, uu_mean (equvalent to **10** for isothermal flows)

Solve momentum equation: using local arrays to store intermediate calculation results (about **100** for isothermal flows, each comsumes 1.1 GB for a 512^3 mesh)

Solve Poisson equation: up to **20** intermediate arrays depending on the boundary conditions used and whether the mesh is stretched

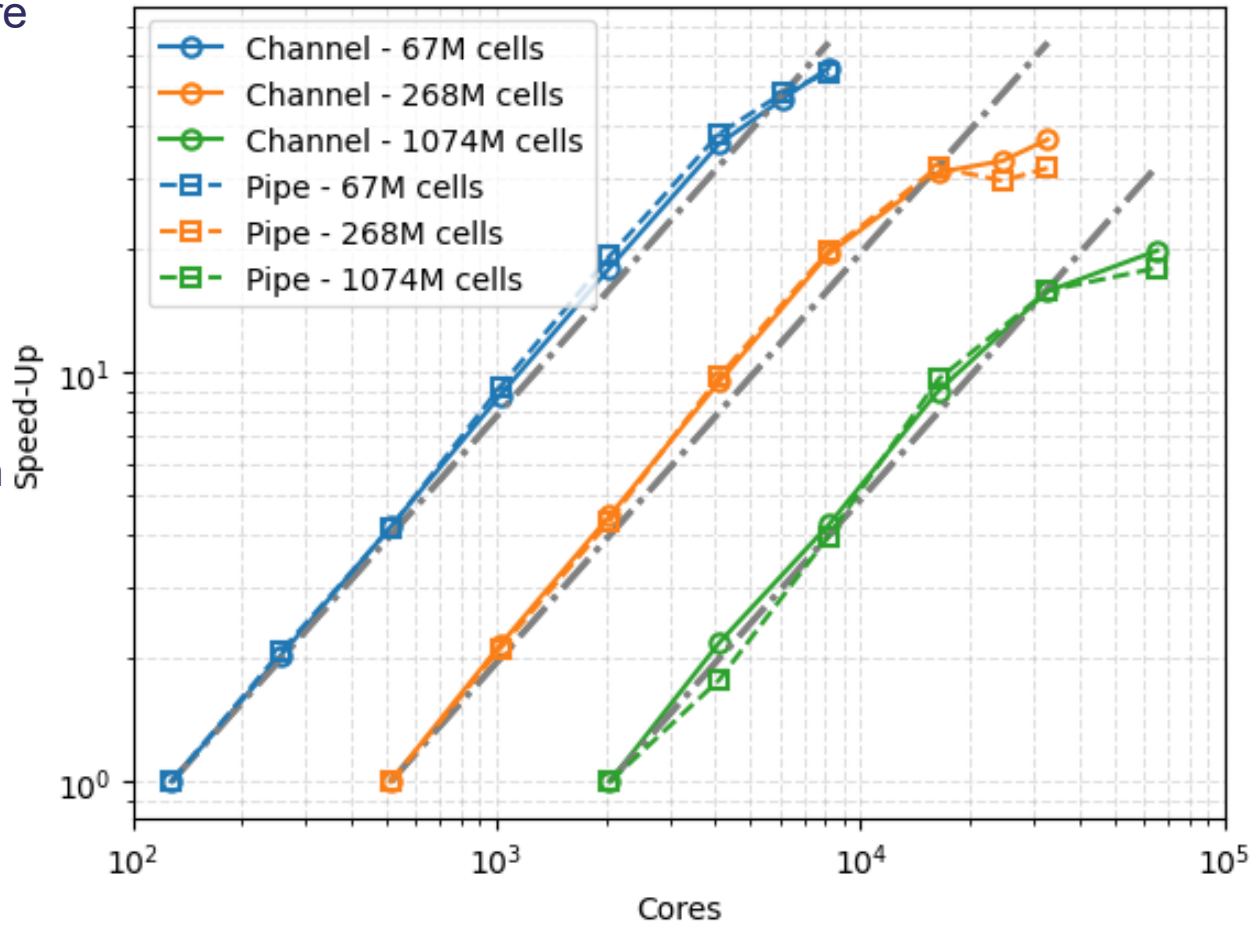
📌 **Key outcome:** Handling a **512^3 (134M)** mesh on a 40 GB GPU memory

Scaling on CPUs

Strong Scaling

- Excellent (super-linear) scaling at low–moderate core counts
  - Driven by cache efficiency and improved memory locality
- Scaling limit: Knee point at ~16k cells per core
  - Consistent for both cartesian and cylindrical configurations
- Beyond the knee, performance degrades
  - Dominated by global MPI all-to-all communication in 2decomp&FFT lib

Cells	Knee Cores	Knee Cells/core
67M	6k–8k	**~8k–16k**
268M	16k	**~16k**
1B	32k	**~16k–32k**



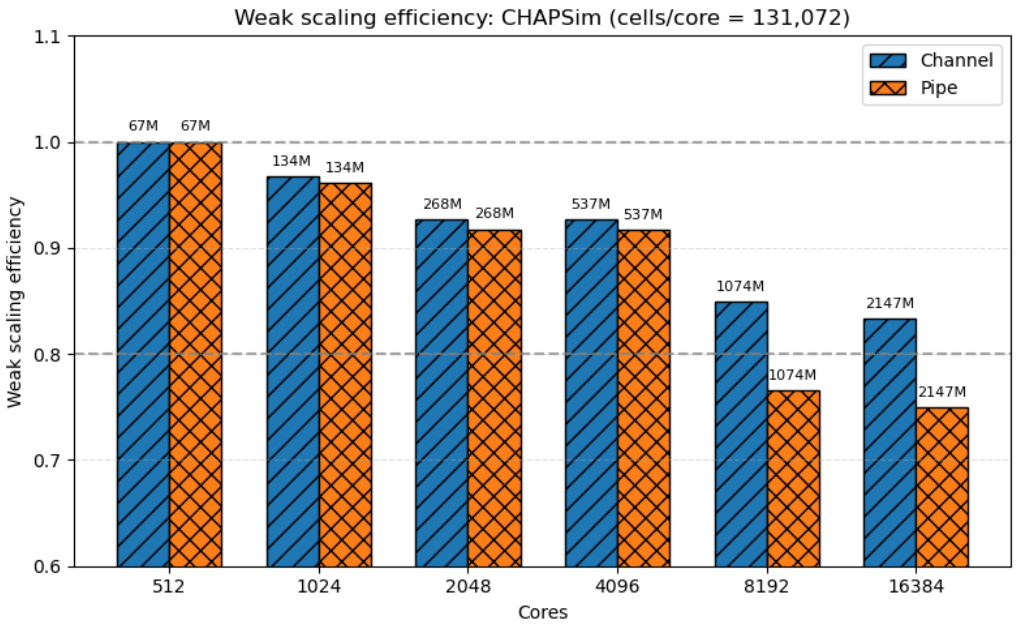
Scaling on CPUs

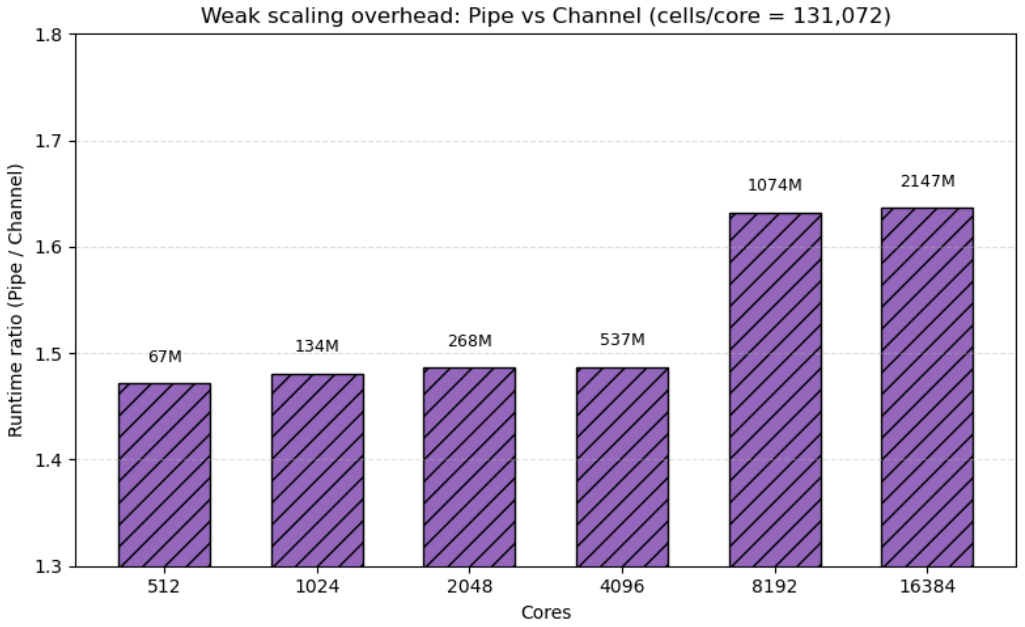
Weak scaling

- Workload = 131,072 cells/core
- Weak scaling efficiency decreases slightly with scale (communication overhead increase)
- Scaling degradation becomes noticeable beyond ~10k–16k cores
  - Channel: ~1.0 (512 cores) → ~0.83 (16,384 cores)
  - Pipe: ~1.0 (512 cores) → ~0.75(16,384 cores)

Cylindrical vs Cartesian

- Cylindrical coord. shows higher sensitivity to scales (extra operations)
  - the TDMA solver introduces a directionally constrained operation that does not scale as efficiently as the fully spectral 3D FFT
- Cylindrical coord. exhibits consistently 40% higher cost over channel flow.

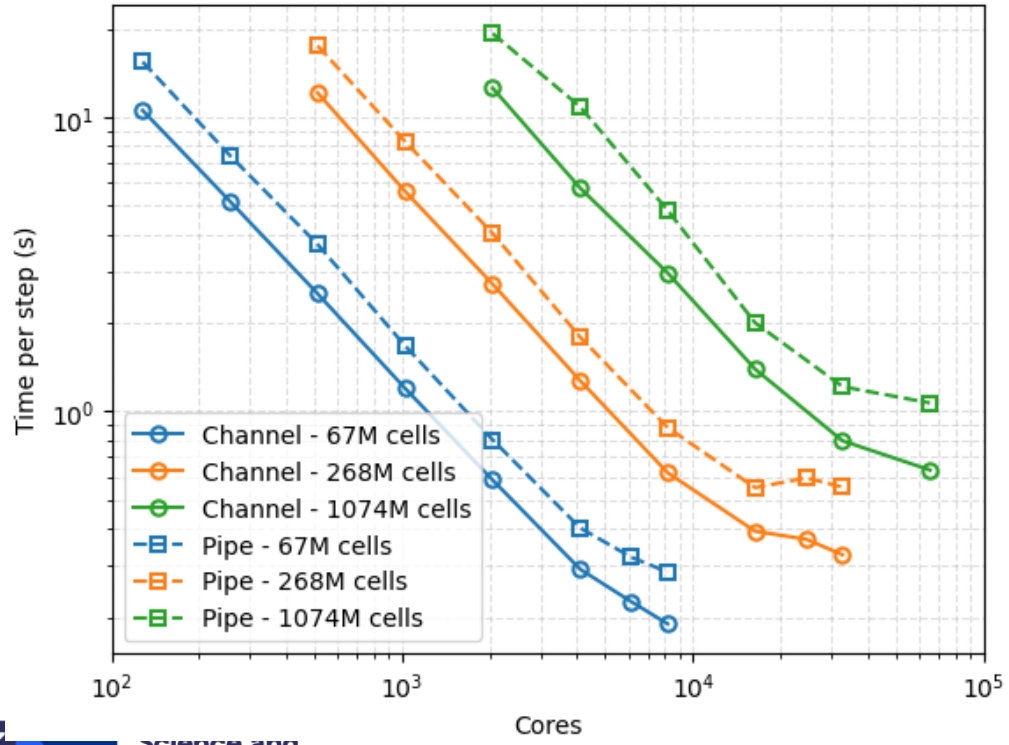


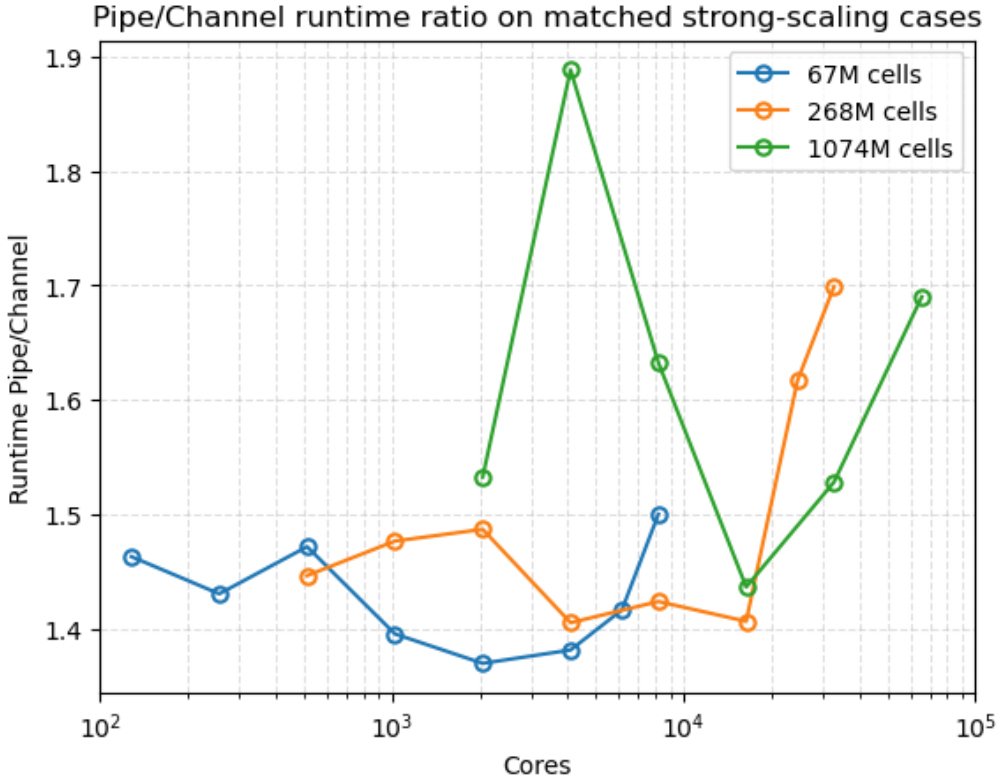


Cylindrical Coordinates Overhead

▪ Strong Scaling

- Pipe (cylindrical) is consistently ~1.4–1.7× slower than Channel (cartesian)
  - Coupling effects from extra geometric terms (not due to 2D-FFT treatment)
- Cylindrical configuration is slightly less scalable than Cartesian





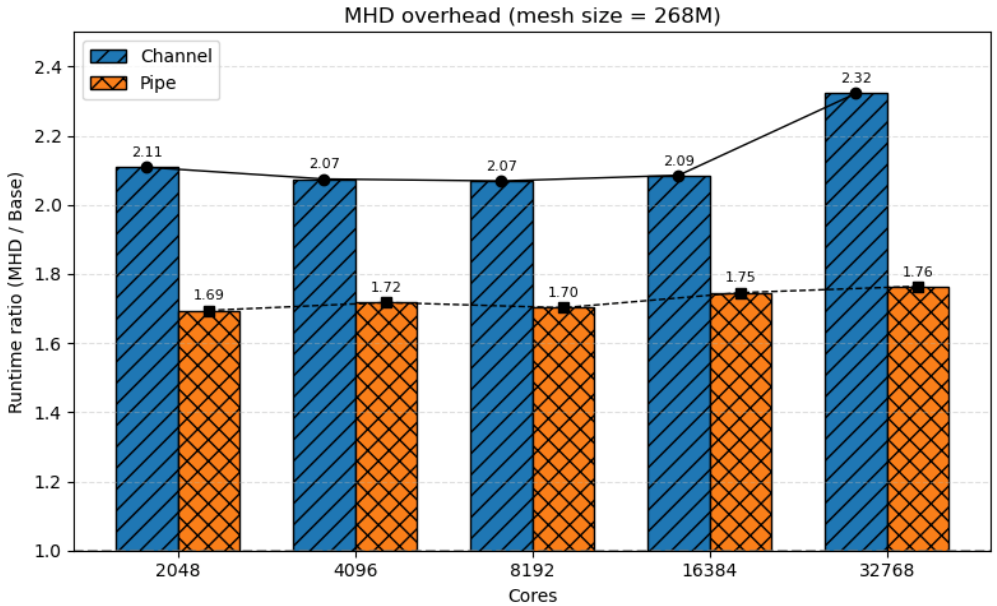
MHD Overhead

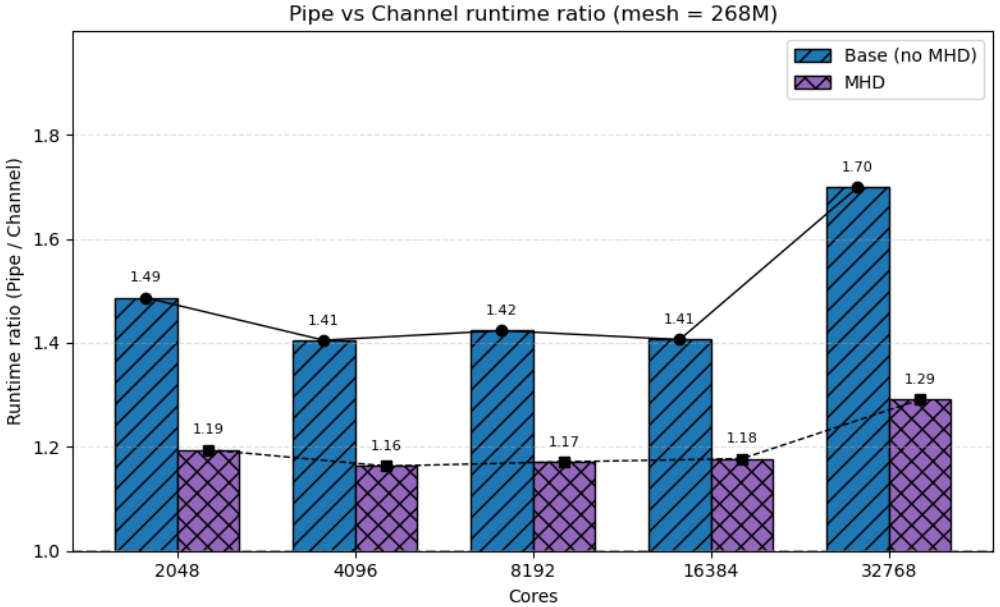
Overall Cost Impact

- MHD introduces significant computational overhead
  - Channel (Cartesian): ~2.0–2.3× increase in runtime
  - Pipe (cylindrical): ~1.7–1.8× increase
  - Higher overhead in Channel due to additional 3D FFT-based Poisson solver, Lower overhead in Pipe using 2D FFT + TDMA, reducing communication cost
  - 2D FFT + TDMA has lower cost than 3D FFT, but poor weak scaling

Cartesian vs Cylindrical

- For no-MHD, Pipe is ~1.5× slower than Channel, but only ~1.2× slower with MHD.
- Under MHD, 3-D FFT communication cost dominate, making solver-specific differences less significant.





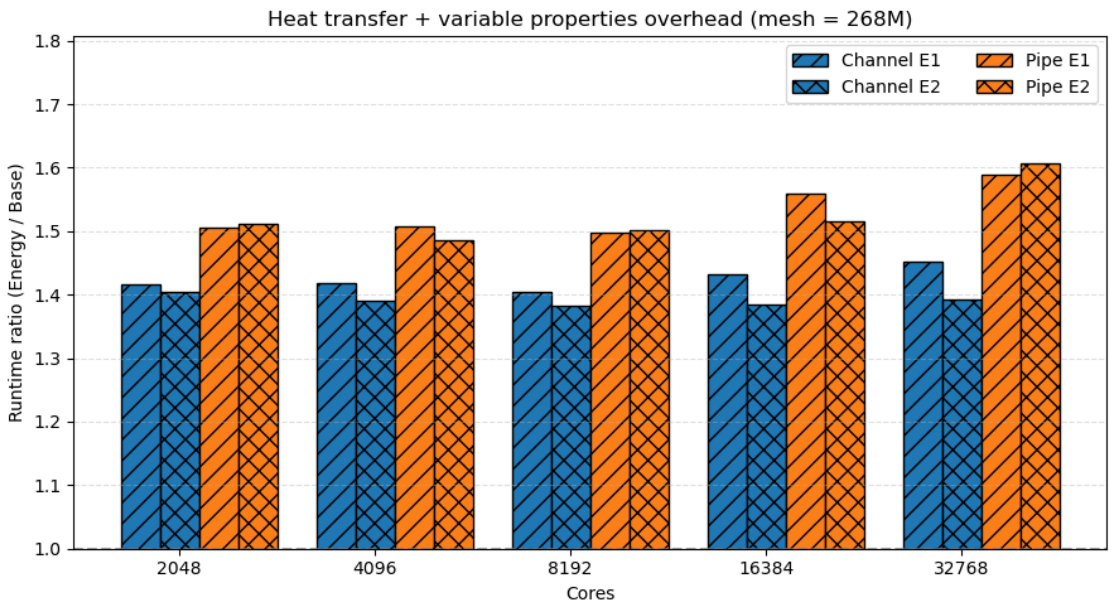
Heat transfer overhead

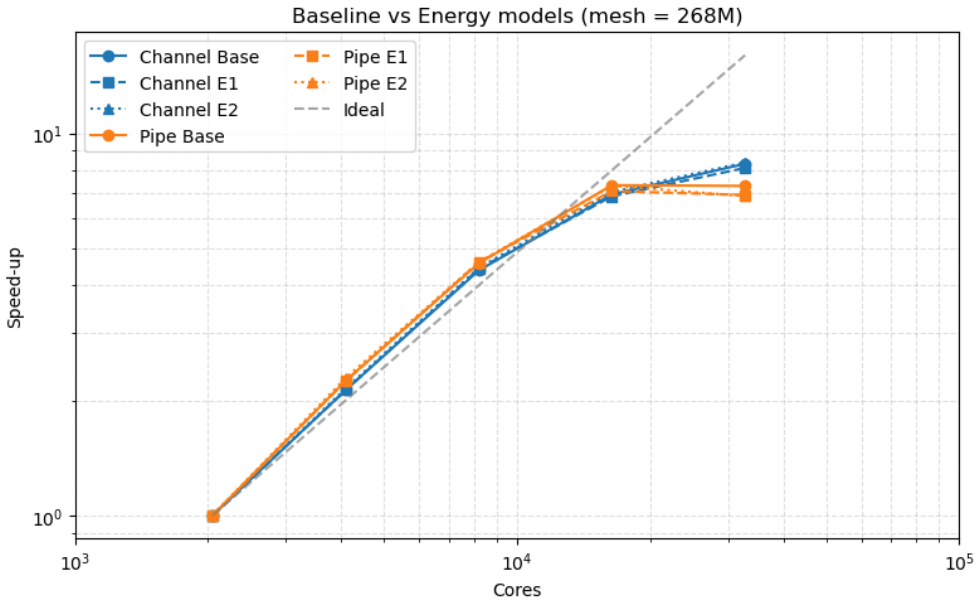
Overall Cost Impact

- Energy equation introduces a consistent ~1.4–1.6× overhead
  - Channel: ~1.38–1.45×
  - Pipe: ~1.50–1.60×
- Thermal property update: Lookup (E1) vs Analytical (E2):
  - very small difference (<5%)
- Cartesian vs Cylindrical
  - Pipe shows slightly higher overhead (~+10–15%)

Scaling Behaviour

- Base and energy cases scale almost identically
- Energy coupling increases cost but does not degrade parallel scalability

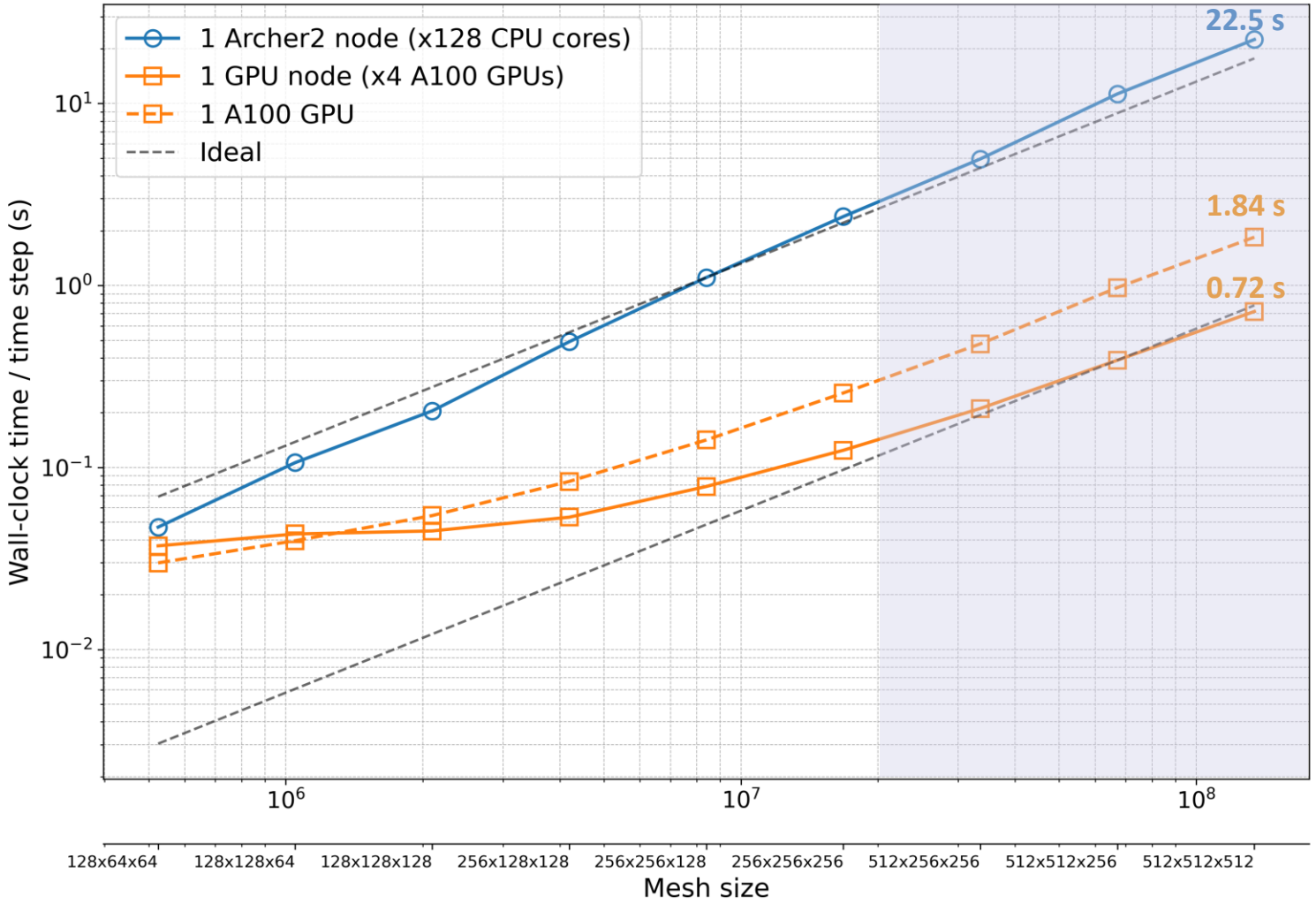




Speedup on GPUs

- **Single GPU node**

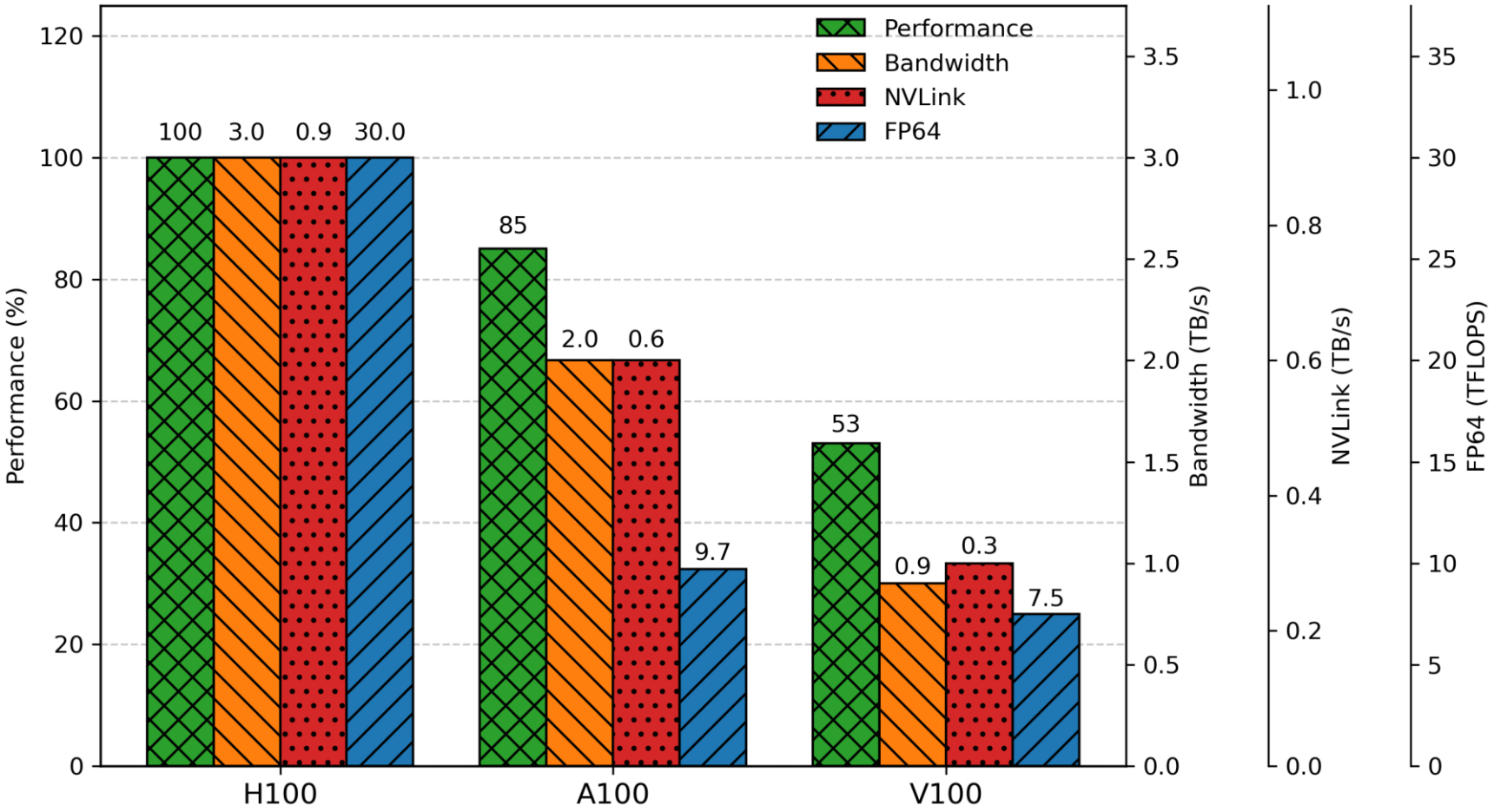
- One NVIDIA A100 node (4 GPUs) provides compute power equivalent to roughly **30 - 40 ARCHER2 nodes**
- Optimal acceleration is observed for workload larger than **10 - 20 million** grid points per GPU



Performance on different GPUs

- **H100 | A100 | V100**

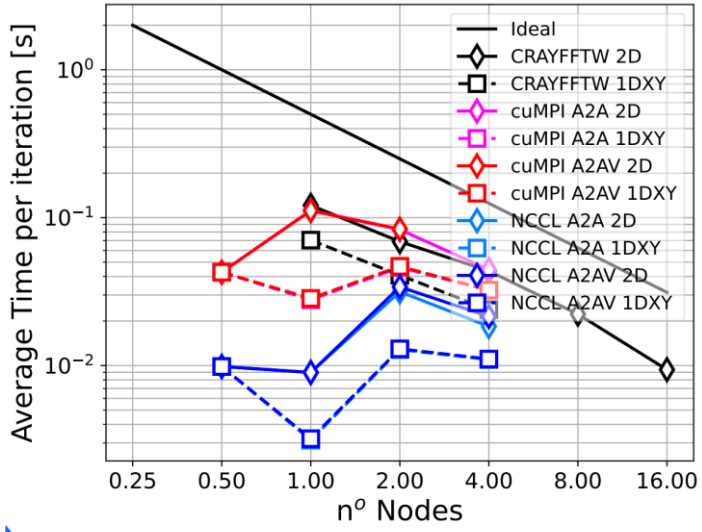
- Performance is limited by memory bandwidth and/or communication, rather than raw compute FLOPs:
  - Numerous small to medium-sized kernels
  - Relatively low compute intensity per kernel
  - Irregular memory access patterns
  - Frequent communication overhead

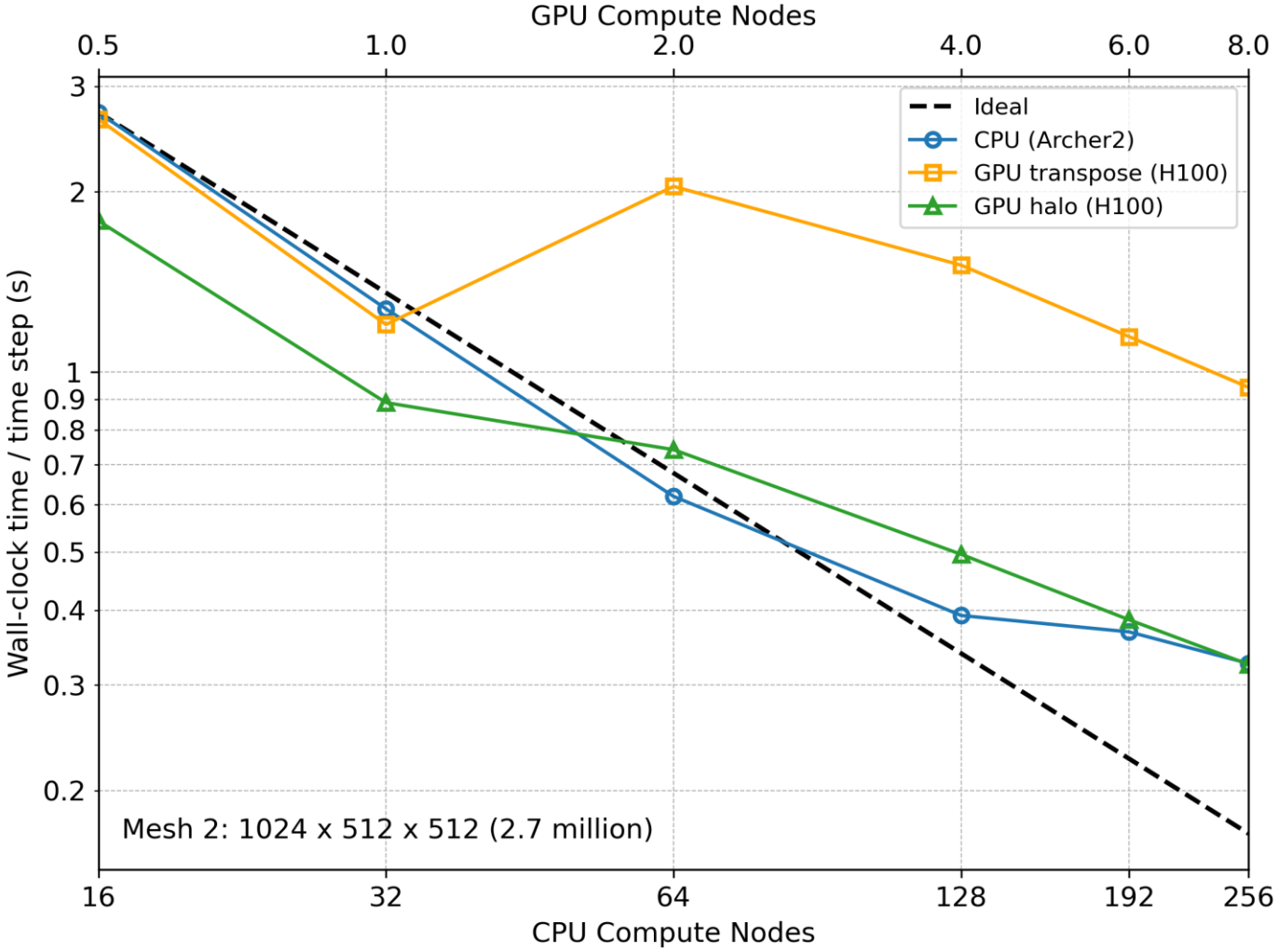


Scaling on GPUs

Results

- Excellent intra-node scaling
- Limited by node-to-node communication, cross-node scaling:
  - Degrades significantly when using transpose with negative efficiency from 1 node to 2 nodes
  - Much better when using halo exchange for calculating the RHS

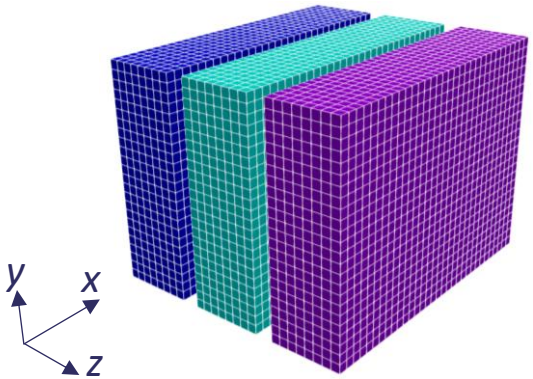


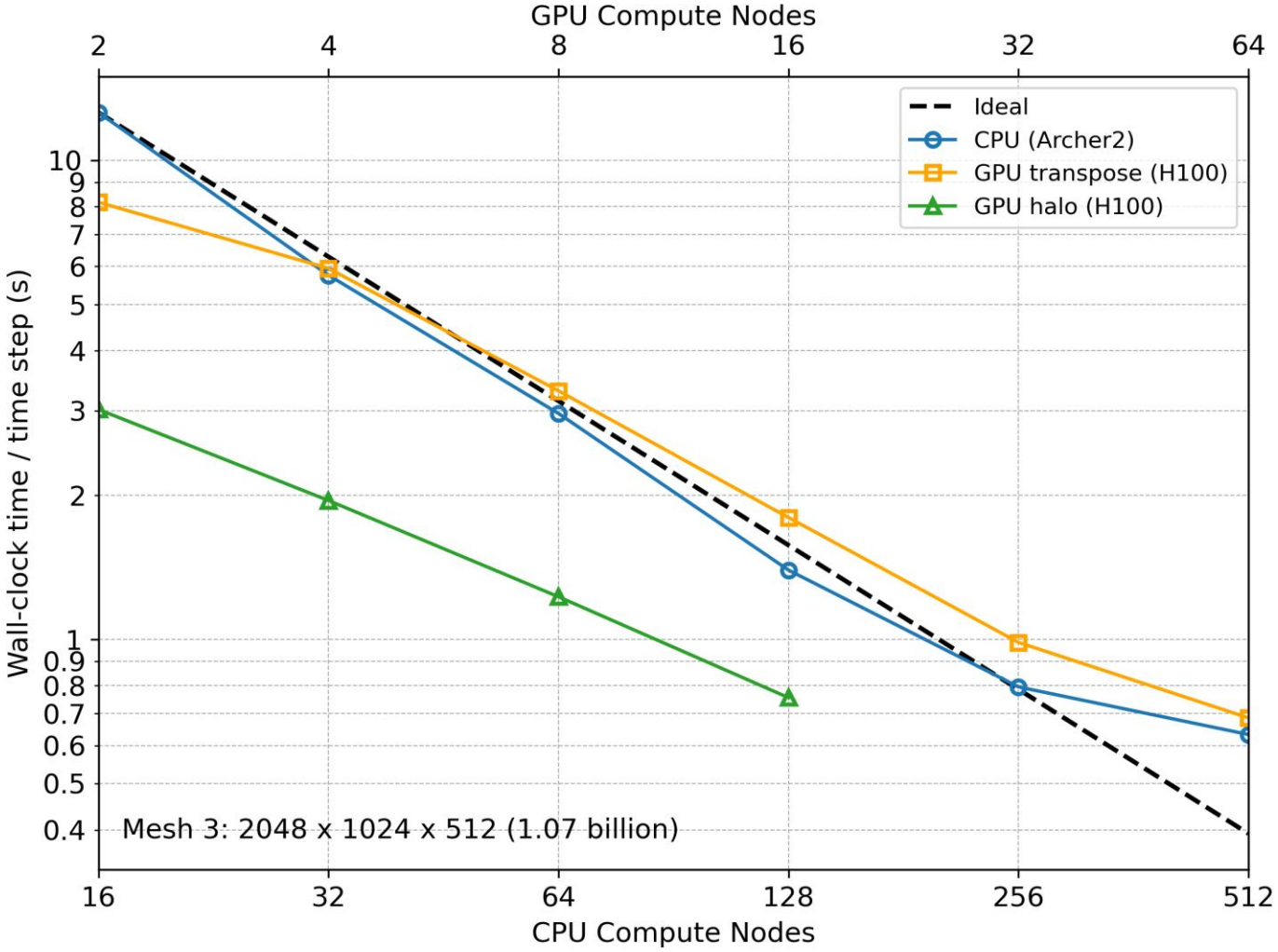


Large mesh

- **Results**

- For a ~1 billion grid point mesh:
  - The GPU-enabled version scales consistently up to 32 nodes (128 GPUs)
  - Efficiency drops at 64 nodes (256 GPUs), due to overly small mesh partitions (4.2 million grid points per GPU)
  - Using halo exchange for calculating the RHS significantly improves cross-node speedup and scaling

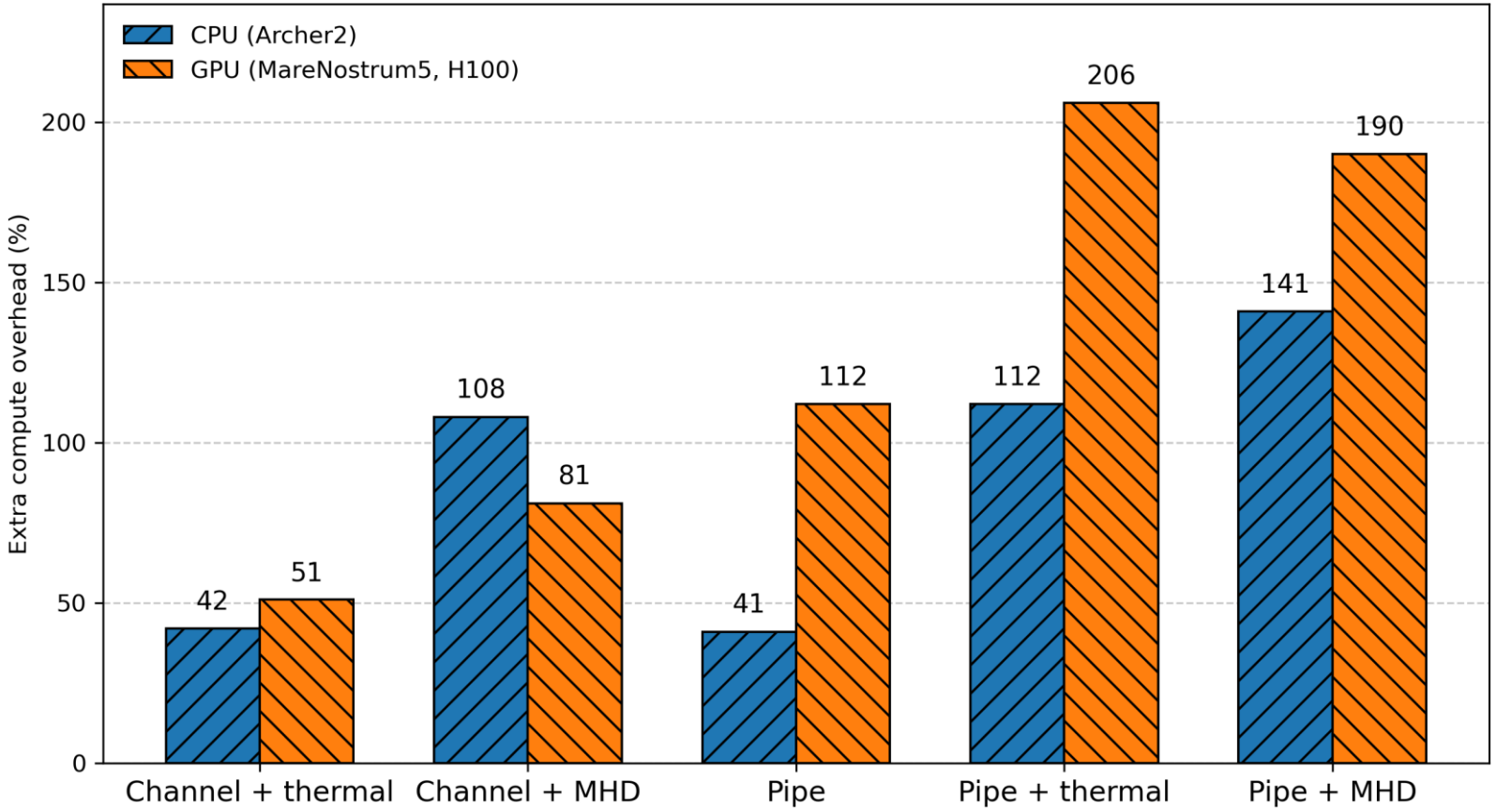




Extra compute overheads

- **Cylindrical coordinates, energy & MHD**

- Compared against the CPU-only version:
  - Significantly higher additional overhead for cylindrical coordinates
  - Similar additional overhead for heat transfer
  - Slightly lower additional overhead for MHD



Extra compute overheads relative to the base case (iso-thermal channel)

Summary

- CHAPSim2 has been ported to NVIDIA GPUs using OpenACC off-loading:
  - **Speedup:** ~30 - 50× speedup on a single A100/H100 GPU node compared to an ARCHER2 CPU node
  - **Scaling:** Intra-node scaling is near-ideal, while cross-node scaling degrades due to node-to-node communications. Using halo-exchange significantly mitigates such effects by reducing size of data movement.

- A detailed HPC performance evaluation has been conducted for CHAPSim2 on ARCHER2:
  - **Strong scaling:** Excellent performance at low-to-moderate core counts (up to 32,768 cores)
  - **Weak scaling:** Efficiency remains high, maintaining around 80% up to 16,384 cores
  - **Coordinates effect:** Simulation in cylindrical coordinates is 40 - 70% slower than the Cartesian counterparts and scales less ideal
  - **Additional physics**: heat transfer introduces a 40 - 60% overhead and MHD adds 70 - 130% relative to the baseline solver